

# OS LAB-7

**Name:** Aditya Kumar

**Registration no:** AP24110010333

**BTech CSE – C**

**Implement the Highest Response Ratio Next (HRRN) CPU scheduling algorithm in C. HRRN is a non-preemptive scheduling algorithm that selects the process with the highest response ratio, balancing short jobs with long-waiting jobs and addressing the starvation problem inherent in Shortest Job First (SJF).**

**Highest Response Ratio Next (HRRN) Scheduling:**

**Code :**

```
#include <stdio.h>

#define MAX_PROCESSES 100

typedef struct {
    char name[20];
    int pid;
    int arrival_time;
    int burst_time;
    int completion_time;
    int turnaround_time;
    int waiting_time;
    int response_time;
    int start_time;
    int completed;
} Process;

double response_ratio(Process *p, int current_time)
{
    int waiting_time = current_time - p->arrival_time;
    return (double)(waiting_time + p->burst_time)
        / p->burst_time;
}

int select_process(Process p[], int n, int current_time)
{
    int selected = -1;
    double highest_rr = -1.0;

    for (int i = 0; i < n; i++) {
        if (p[i].completed == 0 &&
            p[i].arrival_time <= current_time) {
            double rr = response_ratio(&p[i], current_time);

            if (selected == -1 || rr > highest_rr) {
```

```

        selected = i;
        highest_rr = rr;
    }
    else if (rr == highest_rr) {
        if (p[i].burst_time < p[selected].burst_time) {
            selected = i;
        }
        else if (p[i].burst_time == p[selected].burst_time) {
            if (p[i].arrival_time < p[selected].arrival_time) {
                selected = i;
            }
            else if (p[i].arrival_time ==
                    p[selected].arrival_time &&
                    p[i].pid < p[selected].pid) {
                selected = i;
            }
        }
    }
}
}
return selected;
}

void print_gantt_chart(Process p[], int order[], int count)
{
    printf("\nGantt Chart:\n\n");
    printf("|");
    for (int i = 0; i < count; i++) {
        printf(" %s |", p[order[i]].name);
    }
    printf("\n");
    printf("%d", p[order[0]].start_time);
    for (int i = 0; i < count; i++) {
        printf(" %d", p[order[i]].completion_time);
    }
    printf("\n");
}

void display_results(Process p[], int n, int context_switches){
    double total_tat = 0;
    double total_wt = 0;
    double total_rt = 0;

    printf("\n");
    printf("Process | Arrival | Burst | Completion | Turnaround | Waiting | Response\n");
    printf("-----\n");
    for (int i = 0; i < n; i++) {
        printf("%-7s | %-7d | %-5d | %-10d | %-10d | %-7d | %-8d\n",
            p[i].name,
            p[i].arrival_time,
            p[i].burst_time,
            p[i].completion_time,
            p[i].turnaround_time,
            p[i].waiting_time,
            p[i].response_time);

        total_tat += p[i].turnaround_time;
        total_wt += p[i].waiting_time;
        total_rt += p[i].response_time;
    }

    printf("\n");
}

```

```

    printf("Average Turnaround Time : %.2f\n", total_tat / n);
    printf("Average Waiting Time : %.2f\n", total_wt / n);
    printf("Average Response Time : %.2f\n", total_rt / n);
    printf("Total Context Switches : %d\n", context_switches);
}

void hrrn(Process p[], int n)
{
    int current_time = 0;
    int completed = 0;
    int order[MAX_PROCESSES];
    int order_count = 0;
    int context_switches = 0;

    while (completed < n) {
        int index = select_process(p, n, current_time);

        if (index == -1) {
            int next_arrival = -1;
            for (int i = 0; i < n; i++) {
                if (p[i].completed == 0) {
                    if (next_arrival == -1 ||
                        p[i].arrival_time < next_arrival) {
                        next_arrival = p[i].arrival_time;
                    }
                }
            }
            current_time = next_arrival;
            continue;
        }

        if (order_count > 0) {
            context_switches++;
        }

        order[order_count] = index;
        order_count++;

        p[index].start_time = current_time;
        p[index].response_time =
            p[index].start_time - p[index].arrival_time;
        current_time += p[index].burst_time;
        p[index].completion_time = current_time;
        p[index].turnaround_time =
            p[index].completion_time -
            p[index].arrival_time;
        p[index].waiting_time =
            p[index].turnaround_time -
            p[index].burst_time;
        p[index].completed = 1;
        completed++;
    }

    print_gantt_chart(p, order, order_count);
    display_results(p, n, context_switches);
}

int main()
{
    Process p[MAX_PROCESSES];
    int n;

```

```

FILE *file;

file = fopen("processes_hrrn.txt", "r");
if (file == NULL) {
    printf("Error: Could not open processes_hrrn.txt\n");
    return 1;
}

if (fscanf(file, "%d", &n) != 1) {
    printf("Error: Invalid input file\n");
    fclose(file);
    return 1;
}

if (n <= 0 || n > MAX_PROCESSES) {
    printf("Error: Invalid number of processes\n");
    fclose(file);
    return 1;
}

for (int i = 0; i < n; i++) {
    if (fscanf(file,
                "%19s %d %d",
                p[i].name,
                &p[i].arrival_time,
                &p[i].burst_time) != 3) {
        printf("Error: Invalid process data\n");
        fclose(file);
        return 1;
    }
    p[i].pid = i + 1;
    p[i].completion_time = 0;
    p[i].turnaround_time = 0;
    p[i].waiting_time = 0;
    p[i].response_time = 0;
    p[i].start_time = 0;
    p[i].completed = 0;
}
fclose(file);

hrrn(p, n);
return 0;
}

```

## Output:

```
aditya-forge@LAPTOP-184D82EB:~/OS/week7$ gcc hrrn.c -o hrrn
aditya-forge@LAPTOP-184D82EB:~/OS/week7$ ./hrrn
```

Gantt Chart:

```
| P1 | P3 | P2 | P4 |
0   4   6   12  17
```

| Process | Arrival | Burst | Completion | Turnaround | Waiting | Response |
|---------|---------|-------|------------|------------|---------|----------|
|---------|---------|-------|------------|------------|---------|----------|

|    |   |   |    |    |   |   |
|----|---|---|----|----|---|---|
| P1 | 0 | 4 | 4  | 4  | 0 | 0 |
| P2 | 1 | 6 | 12 | 11 | 5 | 5 |
| P3 | 2 | 2 | 6  | 4  | 2 | 2 |
| P4 | 3 | 5 | 17 | 14 | 9 | 9 |

Average Turnaround Time : 8.25  
Average Waiting Time : 4.00  
Average Response Time : 4.00  
Total Context Switches : 3